Build your personal agent

From repeating a useful prompt to selecting a reusable specialist.

THE MAIN QUESTION

How do I turn a repeatedly useful prompt into a reusable, bounded AI teammate?

API Specialist

A shared role for this project and team.

My Reviewer

A personal role that can advise, not edit.

Not a newly trained model: a role, workflow, toolset and clear boundaries.

A model, instructions, tools and a work loop

Chat is the conversation. An agent can choose steps, use tools and inspect the results.

MODEL: proposes the next step

INSTRUCTIONS + CONTEXT: guide it

TOOLS + PERMISSIONS: limit actions

1

Understand

Read the goal and relevant context.

2

Act

Request an available tool or return advice.

3

Inspect

Read the result; check for gaps.

4

Adjust / stop

Revise the approach or return the result.

Answering: "Here is an approach." Acting: inspect files, make allowed changes, check results.

The same model can power different roles. More autonomy needs clearer boundaries.

Use a specialist when the role keeps recurring

A focused role is easier to explain, evaluate and bound than an "expert in everything".

START WITH WHAT ALREADY EXISTS

Ask	Understand code and answer questions.
Plan	Research, clarify and propose the steps.
Agent	Implement with the available tools.

CUSTOM ROLE = APPROACH + TOOLS + OUTPUT

API Specialist
Plans endpoints using team patterns.

Frontend reviewer
Checks accessibility and UI contracts.

Data reviewer
Checks migrations and data integrity.

Pipeline investigator
Traces failures and reports evidence.

One task: prompt. Shared rules: instructions. Role + tools: agent. Workflow + resources: skill.

Metadata above. Behaviour below.

Use a .agent.md file: YAML frontmatter, then plain Markdown instructions.

```
---
name: Focused Reviewer
description: Check API error handling
tools: ['read', 'search']
---
Review the supplied API code.
Read related context, then check errors.
Report file, line, impact and fix.
Stop after reporting. Do not edit.
```

name + description

Identify the role and when to use it.

tools

Choose the available capabilities.

Markdown body

Role, workflow, rules, output, stop.

Optional model

Choose an engine, not a new role.

Use precise descriptions and valid YAML. Choose supported tools in the editor; verify discovery.

Same kind of role. Different home and audience.

Choose where the configuration belongs before creating it.

Workspace / team

`.github/agents/*.agent.md`

Shared and versioned through Git.
Applies to this repository's workflow.
Review changes like other team code.

LAB A: API Specialist

User / personal

Created in User / profile storage

Not committed to this repository.
For your own recurring workflows.
Availability follows its user / profile setup.

LAB B: my-reviewer

In Codespaces, User means that codespace / profile. It does not automatically mean your laptop.

Inside the API Specialist

A useful specialist describes decisions and a workflow, not just expertise.

```
---
name: API Specialist
description: Plan and build OctoCAT APIs
---
You are the team's API specialist.
Read architecture and API guidance first.
Clarify requirements and edge cases.
Plan models, repositories and routes.
Include migrations, Swagger and tests.
Use shared errors and parameterized SQL.
Return files, endpoints and test approach.
For plan-only requests, stop at the plan.
```

Inputs

Existing architecture and API guidance.

Workflow

Requirements > data > routes > checks.

Conventions

Thin routes, shared errors, safe SQL.

Output today

A Cart API plan; no implementation.

No tools field means inherited/default tools. A "plan only" prompt is not a permission barrier.

Inside My Reviewer: read-only by design

Your role, two useful checks, a clear output contract and a narrow toolset.

```
---
name: my-reviewer
description: Review code without changing files
tools: ['read', 'search']
---
You review <AREA> as a <ROLE>.
Read the file and relevant context.
Check <CHECK 1>, <CHECK 2> and obvious bugs.
Return at most 5 actionable findings:
severity | file:line | problem | suggested fix.
Say if none are found or cannot be verified.
Never edit files or run commands.
If asked to fix, explain the fix instead.
```

Tailored role

Your area and recurring review needs.

Concrete checks

For example: safe SQL or accessibility.

Useful findings

Evidence, location and suggested fix.

Stop condition

Report and explain; never apply fixes.

This is your selectable custom role, not the separate built-in Copilot code-review feature.

Signs guide. Locks limit.

Match the available actions to the role; do not rely on wording alone.

Instruction: "Never edit"

Guidance, even when editing tools exist.

Tools: ['read', 'search']

Leaves editing and terminal tools out.

Read + search	Inspect files and related code.	Reviewer needs this
Editing	Change the workspace.	Leave out
Terminal / commands	Execute scripts, tests or shell actions.	Leave out
External / delegation tools	May fetch data or trigger changes elsewhere.	Inspect; least privilege

Tool restrictions, action approvals and organization policy are different layers. Check them all.

Prove the configuration improves the work

Use evidence and controlled comparisons, not a longer or more confident answer.

Draft + validate

Generate from repository evidence.

Only edit the agent file; max 50 lines.

Check YAML, tools and the picker.

Compare outputs

Same task, prompt, model and files.
Fresh chats; same target.

Check patterns, migrations, Swagger, tests and errors.

Challenge the boundary

Ask it to review a file.
Then: 'Fix the first finding.'

Expect an explanation, no file edits and no terminal activity.

Missing from the picker? Check location, YAML, selected target and discovery settings; try a fresh chat.

Hand work over deliberately, not magically

A handoff changes who works next. A subagent handles a delegated task in a separate context.

Plan

Clarify the work

→

Build

Implement the approved scope

→

Review

Inspect the evidence

PASS A SMALL, EXPLICIT PACKET

Goal and accepted decisions.
The current plan / artifact path.
Scope, constraints and finish checks.

KEEP THE ROLE TRUSTWORTHY

Version and review shared profiles.
Retest after tool, model or target changes.
Do not assume all context travels.

Handoff buttons depend on the target. Fallback: select the next agent and name the saved artifact.

The team's workspace API Specialist

Rebuild a shared agent and compare its Cart API plan with the built-in Agent.

THE REQUEST

POST /cart returns a cartId; CRUD under /cart/:cartId. Plan only: files, endpoints and tests.

1	2	3	4	5
3 MIN	4 MIN	7 MIN	5 MIN	6 MIN
Baseline	**Create**	**Draft**	**Compare**	**Improve**
Built-in Agent. Save its plan.	Workspace > api-specialist.	Use repo guidance. 50 lines; one file.	Same prompt and model.	Compare to main. Borrow one idea.

`.github/agents/api-specialist.agent.md`

Done: the agent is selectable, and you can explain what improved. Do not build the Cart API.

LAB F2 | PART B | 20 MINUTES

Your personal, read-only reviewer

Create it for your area, use it on a real file, then challenge its boundary.

1 CREATE | 4 MIN

Choose User > my-reviewer.

tools: ['read', 'search']

2 PERSONALISE | 5 MIN

Fill your role, area and two checks.

Backend, frontend, data or platform.

3 REVIEW | 6 MIN

Open a file from your area.

"Review the file I have open."

4 GUARDRAIL TEST | 3 MIN

"Fix the first finding."

Expect an explanation, not edits.
Check Source Control and tools.

5 SHARE | 2 MIN

Show your pair your best finding.

Done: both agents are selectable; the reviewer gives useful findings without changing files.